

Rangkuman Tahapan Pengembangan

Sistem TanyaPadi — Backend FastAPI + Flutter

Sesi: 16 Juni 2026

1. Status Sistem RAG (Sudah Selesai)

Berikut komponen RAG yang sudah selesai sebelum sesi ini:

Komponen	Detail	Status
Knowledge base	chunks_2.json — 10 chunk dari 7 dokumen (Las et al. 2018, Pujiharti et al. 2008)	Selesai
Embedding model	intfloat/multilingual-e5-base (dipilih via AP@3 comparison)	Selesai
Dense retrieval	ChromaDB dengan cosine similarity	Selesai
Hybrid retrieval	BM25 + ChromaDB + RRF fusion (top-3 chunks)	Selesai
LLM	Groq (LLaMA 3.3 70B) dan Gemini Flash	Selesai
Evaluasi	30 TC, Precision@3, Recall@3, MRR, BERTScore, Faithfulness, FaithLLM	Selesai

Hasil evaluasi (rata-rata dari JSON output):

Metrik	Dense	Hybrid	Selisih
Precision@3	0.5000	0.6333	+0.1333
Recall@3	0.3883	0.5133	+0.1250
MRR	0.7000	0.8000	+0.1000
BERTScore F1	0.7750	0.7730	-0.0020
Faithfulness (token)	0.8955	0.9501	+0.0546
Faithfulness LLM	0.4098	0.5537	+0.1439
Retrieval Time	548.01 ms	821.13 ms	+273.12 ms

Hybrid dipilih sebagai metode default karena unggul di semua metrik kecuali latency. Trade-off latency (~1.5x lebih lambat) dianggap acceptable untuk konteks pertanian.

2. Temuan Penting dari Analisis RAG

Field sensor yang TIDAK diproses parse_sensor():

Field	Keterangan	Alasan tidak diproses
whm	Kelembaban udara (%)	Knowledge base (chunks) tidak memiliki panduan interpretasi harian yang cukup spesifik
wws	Kecepatan angin (m/s)	Sama — belum ada threshold operasional di knowledge base yang digunakan
st	Suhu tanah (°C)	Sama
sc	Konduktivitas tanah (µS/cm)	Sama

Catatan skripsi: Bukan karena tidak penting, tapi knowledge base yang digunakan (Las et al. 2018 dan Pujiharti et al. 2008) belum cukup mendukung interpretasi keempat parameter tersebut secara ilmiah untuk keputusan

harian. Ini menjadi saran pengembangan di BAB 5.

Konsistensi threshold suhu:

chunk_002 (Dokumen 2) menyebut S1 optimal 24-29 derajat C (kesesuaian lahan). chunk_005 (Dokumen 5) menyebut 22-28 derajat C optimal untuk rekomendasi harian. Notebook dan rag.py mengikuti chunk_005 — konteksnya lebih tepat untuk sistem rekomendasi harian. Tidak ada inkonsistensi, keduanya untuk tujuan yang berbeda.

3. Struktur File Backend (Sudah Dibuat)

File	Fungsi	Status
database.py	Koneksi PostgreSQL (asyncpg), init tabel, seed dummy sensor, get_pod	Selesai
rag.py	init_rag(), parse_sensor(), retrieve_hybrid(), format_context(), build_prompt(), generate()	Selesai
main.py	FastAPI app, lifespan, semua endpoint	Selesai
.env	DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, GROQ_API_KEY, CHROMA_API_KEY_1, CHROMA_API_KEY_2, CHROMA_API_KEY_3, CHROMA_API_KEY_4	Selesai

Struktur folder: main.py / database.py / rag.py / .env / data/chunks_2.json / chroma_db_intfloat_multilingual-e5-base/

4. Endpoint FastAPI (Semua Sudah Selesai)

Method	Endpoint	Fungsi	Body
GET	/health	Cek status server + DB	-
GET	/sensor/latest	Ambil data sensor terbaru dari DB	-
POST	/analyze	Jalankan RAG pipeline, buat sesi baru, simpan ke DB	Kosong
POST	/chat/{session_id}	Kirim pesan lanjutan dalam sesi (dengan konteks sesi sebelumnya)..."} (messages, role="assistant")	
GET	/history	Daftar semua sesi analisis untuk histori percakapan	-

Alur /analyze:

- Ambil sensor terbaru dari DB
- parse_sensor() → sensor_text (7 field)
- retrieve_hybrid(sensor_text) → top-3 chunk
- format_context() → string konteks
- build_prompt(sensor_text, context, time=None, history=None) → messages
- generate(messages, llm='groq') → jawaban LLM
- INSERT chat_session (UUID baru) + INSERT chat_message (role=assistant)
- Return: session_id, jawaban, sensor_parsed, retrieval_time_ms, created_at

Alur /chat/{session_id}:

- Validasi session_id ada di DB
- Ambil sensor dari sesi ini (sensor_id di chat_session)
- Ambil riwayat pesan sesi ini (chat_message ORDER BY created_at)
- retrieve_hybrid(user_message) → top-3 chunk baru

- build_prompt(sensor_text, context, history=riwayat, user_message=pesan_user)
- generate(messages) → jawaban LLM
- INSERT pesan user + jawaban ke chat_message
- Return: session_id, jawaban, history lengkap

5. Skema Database PostgreSQL

Tabel sensor

Kolom	Tipe	Keterangan
id	SERIAL PK	Auto-increment
sm, sph, sn, sp, sk, wtp, wrf	REAL NOT NULL	7 field utama yang diproses parse_sensor()
whm, wws, st, sc	REAL (nullable)	4 field diterima, belum diproses parse_sensor()
time	TIMESTAMPTZ NOT NULL	Waktu pembacaan sensor

Tabel chat_session

Kolom	Tipe	Keterangan
id	TEXT PK	UUID sesi — dibuat saat /analyze dipanggil
sensor_id	INTEGER FK	Referensi ke sensor — untuk ambil konteks sensor di /chat
created_at	TIMESTAMPTZ	Waktu analisis dilakukan

Tabel chat_message

Kolom	Tipe	Keterangan
id	SERIAL PK	Auto-increment
session_id	TEXT FK	Referensi ke chat_session
role	TEXT	'assistant' (jawaban LLM) atau 'user' (pesan petani)
content	TEXT	Isi pesan
created_at	TIMESTAMPTZ	Waktu pesan dikirim

6. Package yang Dibutuhkan

Install sekaligus:

```
pip install fastapi uvicorn asyncpg python-dotenv sentence-transformers chromadb rank-bm25 groq google-genai
```

Package	Fungsi
fastapi	Framework backend REST API
uvicorn	Server ASGI untuk menjalankan FastAPI
asyncpg	Koneksi PostgreSQL asinkron (async/await)
python-dotenv	Baca konfigurasi dari .env
sentence-transformers	Load embedding model intfloat/multilingual-e5-base

chromadb	Vector database untuk dense retrieval
rank-bm25	BM25 index untuk sparse retrieval
groq	Client Groq API (LLaMA 3.3 70B)
google-genai	Client Gemini API

7. Langkah Selanjutnya

Backend — Selesai — endpoint `/health`, `/sensor/latest`, `/analyze`, `/chat/{session_id}`, `/history` sudah jalan dan dites

Flutter UI Home — Menunggu upload UI terbaru dari user — akan dikerjakan bertahap dengan package dio

Flutter UI Chat — Dikerjakan setelah home selesai — konsep sidebar + new chat seperti ChatGPT

Integrasi sensor fisik — Tahap akhir — mengganti dummy data dengan data real dari AWS-003

Dokumen ini merangkum seluruh progress sesi pengembangan TanyaPadi per 16 Juni 2026. Lanjutkan di sesi baru dengan mengacu dokumen ini.